

SUSE Observability

Estudo da implementação e dimensionamento

Helm e Fleet • versão 6 • setembro de 2026

Cinco dashboards, 33 monitores e três charts com configuração compartilhada. Material em português; dashboards e README em inglês.

Use os arquivos Markdown do pacote para copiar comandos sem quebras visuais. Preencha caminhos, contextos, endpoints, nomes de integração e credenciais próprias antes da implantação.

O relatório tests/RESULTADOS.md e a matriz tests/MATRIZ-SOLICITACOES.md distinguem evidências do laboratório, limitações e aceite necessário em outro cliente. Este pacote customizado não substitui o suporte e o dimensionamento oficial do produto.

SUSE Observability: guia de estudo da implementação

Versão 6 - laboratório validado em setembro de 2026. Este documento explica as decisões e ensina a evoluir o projeto. Para executar a implantação, use o procedimento de cliente em `GUIA-HELM-FLEET.md`. Consulte `RESULTADOS.md` para distinguir testes reais de verificações estáticas.

1. O que estamos construindo

Um dashboard apresenta dados; ele não cria os dados. Esta implementação tem quatro etapas: medir na origem, transportar, consultar e interpretar. Um monitor avalia periodicamente uma condição; uma regra de notificação decide quem será avisado. É possível ter um Pod coletor Ready e, ainda assim, nenhum dado chegar ao destino.

O Agent oficial descobre a topologia Kubernetes e coleta recursos dos containers. Os charts complementares acrescentam fontes específicas que não estavam disponíveis com os rótulos e o detalhe necessários. O servidor SUSE Observability recebe as métricas, relaciona recursos à topologia e executa consultas e monitores.

Origem	Coleta	Transporte	Uso
SO do nó	node-exporter + vmagent	Prometheus remote write	Memória disponível, filesystem, CPU e I/O
etcd	vmagent no nó de etcd	Prometheus remote write	Consenso, líder, persistência e quota
Traefik	vmagent e inventário de Ingress/Service	Prometheus remote write	Tráfego por aplicação, códigos, duração e certificados
API Kubernetes	inventário de Pods	Prometheus remote write	Requests, limits e estado de containers regulares
KubeVirt	Prometheus Receiver do Collector OpenTelemetry	OTLP gRPC com TLS	SO convidado, vCPU e disco virtual
Aplicação instrumentada	métricas HTTP da própria aplicação	Pipeline já existente no ambiente	Rota, resultado e duração dentro da aplicação
Arquivos JSON/STY	Jobs de publicação	API SUSE autenticada	Definições de dashboards e monitores

Esses caminhos não são intercambiáveis. O token de ingestão não deve receber permissões administrativas; um Service Token de publicação não instala instrumentação na aplicação.

2. Por que três charts

Um chart por dashboard repetia a URL, credenciais e processo de publicação. Agora o chart central publica as cinco visões em uma instalação. Os coletores permanecem separados porque executam em lugares diferentes: plataforma em cada cluster; KubeVirt no Harvester; conteúdo uma vez no central.

sre-platform-telemetry

`templates/collectors.yaml` cria os DaemonSets habilitados para SO e etcd e suas configurações. `templates/traefik.yaml` cria coleta, descoberta e inventário de backend para os clusters que possuem Traefik. `templates/pod-inventory.yaml` fornece o inventário de configurações de Pods. `files/` contém os pequenos exporters de inventário. `templates/test-traefik.yaml` contém o teste de conectividade Helm.

O node-exporter lê `/proc`, `/sys` e filesystems do host. Os mounts são somente leitura, mas esse acesso continua sensível. O vmagent realiza scrape periódico, filtra famílias, identifica o cluster/nó e envia dados. A

fila local tem limite e não é um arquivo de auditoria durável.

O inventário de Pods usa somente `list pods`, com páginas de 500, teto de 100.000 objetos e validade dos dados. A consulta da API evita a lacuna observada em Pods hotplug. Ele não lê Secrets nem variáveis de ambiente; os requests/limits não são inferidos do consumo.

sre-kubevirt-telemetry

`templates/_config.tpl` monta a configuração do Collector; `templates/resources.yaml` contém Deployment, ConfigMap, ServiceAccount e RBAC. O Receiver Prometheus descobre EndpointSlices em `harvester-system`, usa a CA interna KubeVirt e valida o SAN de cada serviço. Os processadores limitam memória, acrescentam identificação do cluster e agrupam lotes. O exporter envia OTLP gRPC comprimido ao servidor.

O Collector não modifica as VMs, não instala guest agent e não configura o armazenamento. Alguns indicadores só existem quando a VM fornece estatísticas. A memória observada pelo hipervisor não substitui automaticamente o MemAvailable do Linux convidado.

suse-observability-content

`dashboards/*.json` contém as cinco definições completas. `monitors/*.sty` contém as regras SUSE. `files/catalog.json` permite selecionar monitores. `files/panel-monitor-coverage.json` liga cada gráfico à orientação operacional. `files/publisher.py` implementa a publicação e a verificação do conteúdo persistido.

Os dois Jobs são hooks `post-install`, `post-upgrade`, têm prazo máximo, recursos limitados e Service Tokens diferentes. Eles terminam após publicar. Não precisam montar o token Kubernetes do ServiceAccount. Antes de substituir um dashboard, verificam propriedade/identidade e limitam a operação aos objetos gerenciados. A recriação pode alterar IDs; reabra pela lista, não por um bookmark antigo.

Não há Job de SMTP ou notificações. O operador configura SMTP globalmente e associa os monitores a canais pela interface, conforme o procedimento acordado.

3. Anatomia de um Helm chart

Um chart é um pacote de templates Kubernetes, valores e metadados. Helm não inventa APIs no destino: se o template usa uma API não instalada, o deploy falha.

<code>meu-chart/</code>	
<code>Chart.yaml</code>	nome, versão do pacote e versão da aplicação
<code>values.yaml</code>	defaults
<code>values.schema.json</code>	validações de tipos, enumerações e campos obrigatórios
<code>templates/</code>	recursos Kubernetes renderizados
<code>files/</code>	arquivos estáticos usados pelos templates
<code>config/</code>	valores específicos do ambiente
<code>fleet.yaml</code>	seleção e configuração quando Fleet gerencia o pacote

`Chart.yaml.version` identifica a versão do pacote. `appVersion` é informação da aplicação; não faz upgrade automático de uma imagem. Imagens são escolhidas pelos valores dos templates. Tags podem mudar no registry: valide e fixe digests na produção.

`values.yaml` deve ter defaults compreensíveis. Um arquivo passado por `-f` substitui campos; o último arquivo tem precedência. Por isso, no Fleet, a ordem é defaults primeiro e perfil depois. Não duplique todos os defaults em cada perfil. Evite uma variável para cada detalhe que nunca muda: exponha somente escolhas operacionais necessárias.

Um ConfigMap contém configuração não secreta. Um Secret contém material sensível em base64, que não é criptografia. Não version tokens, senhas, kubeconfigs ou Secrets preenchidos. Referencie Secrets existentes no namespace em que o Pod será criado.

Exercício: criar um chart simples

Use um diretório de estudo separado, sem instalar automaticamente no cliente:

```
helm create exemplo-telemetria
helm lint ./exemplo-telemetria
helm template exemplo ./exemplo-telemetria > /tmp/exemplo-renderizado.yaml
```

O scaffold cria muitos recursos genéricos. Remova os que não pertencem ao objetivo, revise imagem, probes, recursos, conta de serviço e exposição de rede. Não use o scaffold sem revisão. Para aprender, altere um valor por vez e compare o manifesto renderizado.

Um campo pode ser obrigatório no template:

```
metadata:
  name: {{ .Release.Name }}-config
  namespace: {{ .Release.Namespace }}
data:
  clusterName: {{ required "clusterName is required" .Values.clusterName | quote }}
```

Depois use `values.schema.json` para rejeitar tipos inválidos antes do deploy. `helm template` prova renderização, não conectividade, autenticação, image pull ou funcionamento do backend.

Hooks são úteis para uma ação finita, como publicar conteúdo externo. Kubernetes/Fleet não observam continuamente o objeto criado fora do cluster. Se alguém editar um dashboard no SUSE, o próximo Job pode reaplicar o arquivo; não existe reconciliação contínua da API externa nesta solução. Incremente `lifecycle.revision` em Git para provocar nova publicação quando necessário.

4. Configuração por camada

Servidor central

Instale e dimensione o servidor pela documentação oficial. Configure DNS/TLS, autenticação, retenção, SMTP e a entrada OTLP. O chart de conteúdo utiliza a URL da interface/API; o Collector KubeVirt utiliza o endpoint OTLP gRPC. Não são a mesma rota.

Um fragmento de `values` de Ingress deve ser acrescentado ao conjunto completo que administra a release do servidor. Não aplique um upgrade apenas com o fragmento e perca sizing, armazenamento ou autenticação. Alterar o servidor pode reiniciar componentes; prepare janela e rollback.

Clusters RKE2

Confirme primeiro qual ingress controller existe. No Rancher-provisionado, a origem da configuração pode ser `spec.rkeConfig.chartValues.rke2-traefik`. No local/importado, pode ser um `HelmChartConfig` em `kube-system`. Edite a fonte correta e preserve campos existentes; não crie dois donos concorrentes.

Traefik precisa expor métricas internamente, com rótulos de serviços. O chart complementar só coleta o endpoint; não habilita o endpoint no processo Traefik. A porta 9100 não precisa ficar pública. Uma `NetworkPolicy` aditiva só deve ser criada depois de avaliar as políticas existentes: a primeira policy que seleciona um Pod pode isolá-lo.

O coletor etcd lê `127.0.0.1:2381/metrics` no próprio nó. A API de clientes do etcd é outra finalidade. O desenho testado não exige publicar 2379/2381 na rede externa. Confirme endpoint e seleção dos nós antes de habilitar a opção; não copie rótulos de outro cliente sem inspeção.

Harvester

Harvester inclui KubeVirt e outros componentes que não existem em um cluster de aplicações comum. Mantenha Traefik desligado quando o ambiente usa NGINX. Instale o Agent pelo processo compatível com a versão, evitando duplicar o add-on e uma release manual.

A CA KubeVirt é pública; copiá-la não fornece chave privada. Ela permite verificar a identidade do endpoint. Sua rotação exige atualizar a cópia e recriar o Pod do Collector. Confiar no certificado no Chrome não configura a confiança do container.

O Secret de registry autentica o download da imagem. O Secret de ingestão autentica o envio de métricas. São credenciais com funções e ciclos de vida diferentes.

5. Como escolher métricas

Comece pela pergunta que alguém precisa responder. “A aplicação está atendendo bem?” pede tráfego, erros e duração. “Por que ficou lenta?” pede correlação com saturação, filas, CPU, memória, disco e dependências. CPU alta sem erro ou latência não é, sozinha, indisponibilidade.

Os quatro sinais do Google SRE são latência, tráfego, erros e saturação. RED aplica Rate, Errors e Duration a serviços. USE organiza Utilization, Saturation e Errors por recurso. São formas de organizar perguntas; não são uma lista universal de gráficos obrigatórios.

Tipo	Interpretação	Operações comuns
Counter	Total acumulado, pode reiniciar com o processo	rate/increase, depois agregação
Gauge	Estado observado: bytes livres, fila, réplicas	valor atual, max/min, tendência
Histogram	Contagens acumuladas por faixas de duração	rate dos buckets, agregação por le, histogram_quantile

Nunca use o valor bruto de um counter como “requisições hoje”. Não calcule média de percentis entre réplicas. Para p95 conjunto, agregue os buckets compatíveis, preservando le, antes de calcular o percentil.

A média é soma das durações dividida pelo número de chamadas. Poucas chamadas extremamente lentas podem elevá-la acima do p95; isso não é matematicamente impossível. Já p50 maior que p95 para a mesma distribuição exige investigar consulta, séries ou apresentação.

Rótulos são dimensões, não texto livre. Cluster, namespace, serviço, rota normalizada e status HTTP costumam ser úteis. Request ID, usuário, URL completa e mensagens de exceção pertencem a logs/traces; como labels, podem multiplicar séries sem controle.

6. Como construir um dashboard neste projeto

1. Escolha a pergunta e a fonte. Consulte a métrica real na instância, veja rótulos, unidade e frequência. Compare uma amostra com kubectl, endpoint bruto ou SO.
2. Teste a consulta no Metrics Explorer para Everything, um cluster e um namespace. Confirme que um cluster inexistente não ganha um falso zero.
3. Abra um dashboard de estudo na interface, adicione o widget e escolha unidade/título. Exporte a definição com a mesma versão da CLI usada no ambiente.
4. Use uma definição existente em dashboards/ como referência de schema. Preserve o formato JSON; nesta base, converter para YAML podia transformar a chave y em booleano.
5. Adicione o painel e sua referência ao layout. Confira x/y/width/height, sobreposição, altura do texto e legibilidade em resolução de trabalho.
6. Dê um nome curto e inglês claro. No “?”, explique o que mede, unidade, janela, escopo, fonte, quando investigar e limitações. “Average response time” é mais didático que “mean latency”.
7. Cadastre a cobertura em panel-monitor-coverage.json. Nem todo gráfico precisa gerar alerta; declare quando é apenas contexto.
8. Execute os testes, renderize o chart e publique em canário. Exporte novamente e compare a definição persistida, não apenas a mensagem de sucesso do comando.
9. Confira cada gráfico e filtro no navegador. API correta não garante legenda correta, nome legível ou layout sem vazios.

Filtros e valores temporários

A variável usa Everything com regex `.*`; as consultas usam `=~`. O rótulo consultado deve existir naquela fonte: `cluster_name` no remote write pode corresponder a `k8s_cluster_name` após OTLP. Não basta trocar o nome visual do filtro.

Filtros dependentes precisam ser redefinidos depois de trocar o cluster. O backend de aplicações pode ter nome diferente do serviço instrumentado. O inventário de Ingress resolve a relação entre identificador interno Traefik e namespace/Service Kubernetes.

`saveVariables=false` evita persistir seleções exploratórias no dashboard. Alterar layout ou query continua sendo edição real. Esta opção não transforma um dashboard editável em modo somente leitura.

Período e estimativas

Os totais acompanham o seletor superior com incrementos de um minuto e passo fixo. A implementação usa recursos do backend da versão validada; não é PromQL universal. Veja [REFERENCIA-OPERACIONAL.md](#) para a expressão exata e sua portabilidade.

Uma chamada não é amostrada individualmente pelo Prometheus: o contador é observado a cada scrape. Uma série criada já com valor 100 não tem uma amostra anterior que permita reconstruir aqueles 100 eventos. Faça aquecimento e linha de base em testes. Números sem K/M melhoram a leitura, mas não tornam a estimativa um registro de auditoria.

Limitação gráfica identificada

No SUSE 2.10.2 testado, nomes de legendas podiam ser ordenados sem reordenar os índices da coluna Last. A solução remove essa coluna numérica, mantém nomes abaixo e preserva curvas/tooltips. Não “corrija” o valor da métrica para compensar um problema visual. Reavalie a coluna ao atualizar o produto.

7. Como validar CPU, memória e disco

CPU de Pod soma os containers regulares medidos. Um núcleo equivale a 1000 mCPU. Request é reserva para agendamento; limit é teto; uso é consumo medido. Um Pod com dois containers não deve ser contado duas vezes no card de configurações ausentes.

Working set de memória inclui mais que heap da JVM. RSS, working set, cache recuperável, MemAvailable e memória configurada da VM respondem a perguntas diferentes. Compare tempos próximos: leituras do kubelet, scrape e ingestão são assíncronas. Diferencie GB decimal de GiB binário.

No Harvester, o SO do host, virt-launcher/QEMU e SO convidado são camadas distintas. Uma VM configurada com 4 GiB pode mostrar MemTotal menor dentro do Linux. O valor reservado pelo Kubernetes não é a quantidade que a aplicação está consumindo.

Filesystems devem preservar dispositivo, tipo e mount relevante. Bind mounts não são capacidade adicional. `/dev/loop0` pode representar um sistema imutável e não o volume que crescerá com os dados. Observe partições operacionais sem esconder a capacidade de dados do Harvester.

PV/PVC informa capacidade provisionada e binding. Uso real de filesystem depende do kubelet/CSI/guest agent. Um volume Block não tem percentual de filesystem aplicável. Capacidade provisionada, consumo do guest e espaço físico Longhorn com réplicas/snapshots não são equivalentes.

CPU scheduler delay do KVM indica espera para executar; não use automaticamente o nome VMware CPU Ready. Latência média de I/O divide tempo acumulado por operações concluídas. Disco sem operações não tem média de latência definida. A fila ponderada do host e a fila de disco virtual não são necessariamente a mesma medida.

8. Como criar e encaminhar um monitor

Copie uma regra semelhante em `monitors/` e crie um `identifier` estável. Defina `query`, `comparação`, `limiar`, `janela`, `severidade`, `recurso da topologia` e `remediation`. Acrescente o mesmo conteúdo ao catálogo. O teste de consistência evita publicar uma `query` enquanto a documentação descreve outra.

O `urnTemplate` precisa resolver um objeto existente. Um monitor pode executar sem erro e ainda produzir `UNMAPPED` se o `identifier` do `Service` estiver errado. Verifique `avaliação agendada` e `associação`, não só `importação`.

Para erros HTTP, esta base exige proporção acima de 5% e volume mínimo de 100 chamadas por janela, durante amostras observadas de cinco minutos. O mínimo evita que uma única chamada gere um alerta de 100%. As regras usam janelas próprias; o seletor do dashboard não muda a avaliação do monitor. Lacunas de dados reduzem o que a janela comprova; disponibilidade da coleta deve ser analisada separadamente.

Um limiar deve indicar ação. Considere `baseline`, `objetivo do serviço`, `horário`, `replicação` e `impacto`. `Disk latency` de 20 ms é um ponto inicial, não um SLA universal para SSD, disco mecânico e armazenamento remoto. Alertas nativos já cobrem diversas falhas de Pods; duplicá-los pode produzir ruído.

Com SMTP global configurado, crie uma notificação manual filtrada pelos monitores ou pela tag `sre-dashboards`. Selecione `Critical` e, quando necessário, `Deviating`. Teste o canal e depois um incidente controlado. O e-mail padrão “Ye Component” é fictício. O aceite real exige recurso correto, mensagem de abertura e recuperação recebida.

A mesma falha pode ser observada na aplicação e no ingress. No laboratório os dois alertas foram intencionais para validar camadas. No cliente, escolha roteamento, agrupamento e responsabilidade para evitar acionamentos duplicados. Não altere `etcd`, disco ou memória para forçar falhas sem plano de exercício aprovado.

9. Segurança, custo e desempenho

Permissões mínimas reduzem superfície, mas não tornam uma integração sem risco. `HostNetwork` e leitura da raiz do nó são acessos sensíveis. Revise `admission policies`, portas, TLS, egress e segredos; não abra uma exceção global de segurança para instalar um coletor.

Use `allowlists` de famílias e retenção proporcional à necessidade. A taxa de ingestão cresce aproximadamente com séries ativas divididas pelo intervalo de scrape; histogramas multiplicam séries pelos buckets e combinações de rótulos. Rotas dinâmicas, novos Pods e discos podem alterar essa ordem de grandeza.

`Requests` e `limits` reservam/capam recursos, mas limites pequenos podem causar `throttling`, `OOM` ou descarte de telemetria. O modo observe desta base não aplica os limites opcionais de amostras/séries; eles precisam de `baseline` antes de `enforce`, pois excedê-los pode falhar um scrape inteiro.

Filas são buffers finitos. A fila `vmagent` é medida em bytes; a fila `OTel` em requests. Reinício, falta de disco, expiração de `retry` ou limite de memória podem perder dados. Alertas que dependem do próprio servidor não substituem uma sonda externa para indisponibilidade do servidor.

Veja `IMPACTO-E-SEGURANCA.md` para a medição no laboratório. Ela não prova impacto zero nem inclui uma comparação A/B sem coletores. Faça canário e compare janelas representativas antes de ampliar. Consulte o `sizing` oficial do servidor para crescimento de ingestão, consultas e retenção.

10. Como evoluir com segurança

1. Faça uma mudança pequena e registre a hipótese, efeito esperado e `rollback`.
2. Valide origem e consulta antes da estética.
3. Execute testes de regressão e `Helm render/lint`; teste políticas no destino.
4. Instale em canário, observe logs, filas, perda de amostras, uso de recursos e fontes.

5. Compare definições persistidas e GUI; teste filtros e períodos.
6. Para alerta novo, valide topologia, conteúdo, abertura, recuperação e destinatário.
7. Versione apenas arquivos públicos essenciais e informe diferenças/limites no changelog.
8. Mantenha responsável por imagens, tokens, CAs, thresholds e compatibilidade de versões.

Dashboards são parte de uma operação SRE. SLOs, orçamentos de erro, probes externos, traces/logs, restauração de backups, escalonamento e resposta a incidentes precisam de decisão do cliente. A existência de uma métrica em outra ferramenta não basta para justificar custo e alerta adicional.

Referências de estudo

- SUSE dashboards:

<https://documentation.suse.com/cloudnative/suse-observability/latest/en/use/dashboards/dashboards.html>

- SUSE monitores: <https://documentation.suse.com/cloudnative/suse-observability/latest/en/use/alerting/k8s-add-monitors-cli.html>

- SUSE permissões: https://documentation.suse.com/en-us/cloudnative/suse-observability/latest/en/setup/security/rbac/rbac_permissions.html

- SUSE notificações: <https://documentation.suse.com/en-us/cloudnative/suse-observability/latest/en/use/alerting/notifications/configure.html>

- Helm estrutura e práticas: https://helm.sh/docs/chart_best_practices/

- Fleet: <https://fleet.rancher.io/0.15/reference/ref-fleet-yaml>

- Prometheus práticas de instrumentação: <https://prometheus.io/docs/practices/instrumentation/>

- Google SRE, quatro sinais: <https://sre.google/sre-book/monitoring-distributed-systems/>

- Grafana, RED: <https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/>

- Datadog, métricas para monitoramento: <https://www.datadoghq.com/blog/monitoring-101-collecting-data/>

As versões e evidências exatas do laboratório estão em RESULTADOS.md. As fontes orientam o desenho; não certificam automaticamente os charts customizados.

Como interpretar os Pods próximos do limite

Os gauges destacam o maior percentual. Para CPU e memória, há uma contagem de Pods acima de 80% do limite e, logo abaixo, a lista somente desses Pods. Uma contagem zero significa que nenhum Pod elegível passou do limiar; falta de dados não é convertida em estado saudável. Apenas Pods com uso e limites positivos em todos os containers regulares são elegíveis. A média é de cinco minutos no fim do período selecionado.

Outros dois gráficos selecionam os cinco maiores consumidores absolutos de CPU (mCPU) e memória (bytes), independentemente de estarem acima de 80%. O grupo é escolhido no fim do período; as curvas mostram seu consumo real ao longo do tempo. Requests e limits dos mesmos Pods ajudam a comparar uso e configuração. Um Pod com 17,5% pode aparecer neste ranking, mas não na lista acima de 80%.

A versão 2.10.2 oferece gráficos de linha, barra, número, gauge e texto nos dashboards, sem widget de tabela nativa. As listas de estado foram compactadas e receberam um link para a consulta no Metrics Explorer. Ative **Table view** para ler nomes e valores em colunas. A tabela exibe o valor bruto da consulta: nos percentuais de limite, 0,8 equivale a 80% e 0,98 equivale a 98%. O link transporta os filtros; para investigar o passado, selecione novamente o período no Explorer. A coluna Last da legenda foi retirada por um defeito de associação entre nomes e valores nessa versão.

80% é um ponto para investigar, não uma recomendação automática de aumentar recursos. CPU requer correlação com throttling e latência; memória requer verificar RSS, cache, crescimento e OOM. Os monitores mantêm suas próprias janelas e critérios. O card de histórico limitado a 24 horas foi removido: confundia uma amostragem limitada com a retenção total do produto.

Comparar um benchmark curto com o dashboard

O navegador calcula a taxa durante a execução do teste. O cálculo de taxa recente (retirado do dashboard para simplificar a leitura) calcula o incremento entre as duas últimas coletas, normalmente 30 segundos. Após o teste, duas coletas sem chamadas novas fazem essa taxa voltar a zero. A média de cinco minutos ainda inclui o teste. Exemplo: 200 chamadas em 1,49 segundo equivalem a cerca de 134 req/s no navegador, 6,67 req/s se concentradas em um intervalo de coleta de 30 segundos, e 0,67 req/s na janela de cinco minutos. Esses números medem janelas diferentes.

Para testar contagens: use uma aplicação de teste, faça uma chamada inicial para cada rota/método/código que será exercitado, aguarde pelo menos duas coletas e a ingestão, registre os counters, execute o teste e aguarde a ingestão antes de comparar os deltas. Uma série nova que aparece inicialmente com valor 10 não prova em qual instante essas dez chamadas aconteceram; rate/irate não podem reconstruir um incremento sem amostra anterior. Não faça esse aquecimento com falhas em aplicações de cliente; limite-o à aplicação de teste. Para hora e quantidade exatas de cada request, use access logs/traces. O p95 do histograma também é uma estimativa por faixas, não o percentil exato calculado pelo navegador.

Recursos, armazenamento e segurança

Medição do laboratório em 11/09/2026. Os números abaixo ajudam a planejar um piloto; não são uma certificação de desempenho nem substituem o dimensionamento oficial do SUSE Observability.

1. O que foi medido

Foram separados três custos: containers adicionais de coleta, Jobs temporários de publicação e servidor central. Os dashboards consultam dados; criar outro gráfico sobre uma série existente não grava outra cópia dessa série. Coletores, novas séries, monitores, consultas simultâneas e retenção podem aumentar o consumo.

Os coletores foram amostrados dez vezes, a cada 30 segundos, durante aproximadamente quatro minutos e meio, pela API `metrics.k8s.io`. CPU representa a taxa de uso na janela informada pelo Metrics Server; memória representa a medida de uso exposta por essa API. Não é um teste de carga máxima. O histórico do servidor foi comparado em janelas de uma hora, no mesmo horário, há um, três e sete dias. O uso dos PVCs veio do kubelet; o armazenamento de métricas também foi conferido diretamente nas estatísticas do VictoriaMetrics.

Não existe uma medição controlada anterior à primeira instalação do projeto. O ponto de sete dias atrás já tinha componentes de observabilidade em execução. Portanto, o aumento histórico do servidor não pode ser atribuído integralmente a estes charts. Caches, tráfego, novos recursos descobertos, saúde, topologia e outras integrações também mudaram.

2. Coletores adicionais nos clusters

Cluster	Nós	CPU média	Maior CPU amostrada	Maior memória amostrada	Requests CPU / memória	Limits CPU / memória
Observability	6	30 mCPU	39,4 mCPU	268,7 MiB	180 mCPU / 568 MiB	2450 mCPU / 2240 MiB
Rancher	3	21,4 mCPU	27,9 mCPU	269,6 MiB	120 mCPU / 400 MiB	1700 mCPU / 1568 MiB
Harvester	1	18,3 mCPU	31,0 mCPU	192,1 MiB	65 mCPU / 360 MiB	950 mCPU / 1184 MiB

Inclui plataforma, inventários e, no Harvester, o Collector KubeVirt: 23 Pods e 38 containers. Exclui o Agent oficial e o servidor SUSE, que já são pré-requisitos. Cada linha é a soma dos containers adicionais naquele cluster. 1000 mCPU = 1 núcleo.

O maior uso observado representou aproximadamente 0,11%/0,23%/0,10% da CPU allocatable e 0,37%/1,13%/0,15% da memória allocatable dos respectivos clusters. Essa proporção depende do tamanho dos nós do laboratório. Requests são reservas utilizadas pelo scheduler; limits são limites por container, não uma reserva adicional nem um consumo contínuo garantido.

Para reproduzir este laboratório, planeje inicialmente as **reservas da tabela**, acrescidas de margem para rollout, outros workloads e picos. Não use a CPU média como request recomendado. Para outro cliente, recalcule as quantidades de nós, membros etcd e clusters.

Valores padrão por container

Componente	Request CPU / RAM	Limit CPU / RAM	Quantidade típica
node-exporter	10m / 24Mi	100m / 96Mi	Um por nó Linux
vmagent	10m / 32Mi	150m / 128Mi	Um por nó; um por membro etcd; um para cada inventário/Traefik habilitado
Inventário de Ingress	5m / 24Mi	100m / 64Mi	Um por cluster com Traefik
Inventário de Pods	5m / 48Mi	100m / 192Mi	Um por cluster
Collector KubeVirt	20m / 192Mi	300m / 512Mi	Um por Harvester
Publicador de conteúdo	20m / 64Mi	200m / 192Mi	Dois Jobs temporários no central
Teste Helm de Traefik	5m / 32Mi	100m / 96Mi	Um Pod temporário quando executado

Todos os containers renderizados têm requests e limits de CPU e memória. Os schemas dos três charts exigem esses campos e rejeitam ausência ou zero. O teste `test_resources.py` verifica todos os perfis e casos inválidos. Kubernetes também valida se requests excedem limits no aceite do manifesto. O pacote não altera os recursos do Agent oficial, Traefik, etcd, aplicações ou servidor SUSE.

A memória interna de trabalho do vmagent é limitada a 64 MiB, dentro do limit de 128 MiB. No OTel, `memory_limiter` usa 384 MiB dentro de um limit de 512 MiB, com margem para picos e processamento. Limites muito baixos podem causar throttling, OOM ou perda de coleta; não foram reduzidos com base na amostra curta.

3. Servidor central: comparação histórica

Momento	CPU média em uma hora	Memória média em uma hora
Sete dias antes	0,657 núcleo	19,524 GiB
Três dias antes	0,811 núcleo	20,265 GiB
Um dia antes	0,815 núcleo	19,872 GiB
Medição atual	0,871 núcleo	20,608 GiB

A diferença observada em relação a sete dias antes foi **+0,214 núcleo e +1,084 GiB de memória** no conjunto do servidor. Isso é comparação de períodos, não uma medição causal do custo dos dashboards. Não significa que todo cliente precise acrescentar exatamente esses valores.

A consulta soma containers do servidor com nomes `suse-observability-*`, excluindo os jobs `suse-observability-content-*`, no namespace central. Usa CPU do Agent em nanocores convertida para núcleos e memória working set convertida para GiB. Compara a média de uma hora em cada data para reduzir a dependência de um único instante.

Para produção, comece pelo perfil oficial compatível com o ambiente observado e acompanhe uso, filas, duração de consultas e ingestão. Os perfis SUSE usam **Default Nodes**: uma máquina grande pode

representar mais de uma unidade de dimensionamento. Não reduza o servidor para os valores baixos de CPU observados neste laboratório. Consulte os [requisitos oficiais](#).

4. Armazenamento: onde houve crescimento

PVC/componente	Uso atual do filesystem	Um dia antes	Sete dias antes	Crescimento em 24h	Capacidade aproximada do filesystem
VictoriaMetrics	0,942 GiB	0,779 GiB	0,184 GiB	0,162 GiB	48,915 GiB
Kafka	38,400 GiB	32,401 GiB	19,567 GiB	5,999 GiB	97,872 GiB
HBase	2,286 GiB	2,004 GiB	1,466 GiB	0,281 GiB	97,872 GiB
Elasticsearch	1,275 GiB	1,179 GiB	0,239 GiB	0,096 GiB	48,915 GiB

Os charts deste projeto **não criam PVCs**. O histórico é armazenado pelo servidor existente. Os números do quadro são do filesystem, que pode ter capacidade útil menor que o tamanho solicitado ao PVC.

O VictoriaMetrics estava configurado com `retentionPeriod=1` (um mês). Não há evidência de necessidade imediata de expandir o seu PVC de 50 GiB por causa deste pacote, considerando o uso atual e esta taxa. Ainda é necessário observar pelo menos um ciclo completo de retenção, crescimento de séries e picos de consulta. Não foi reduzida a retenção nem o tamanho de nenhum volume.

Kafka merece acompanhamento separado

Uma inspeção por diretório encontrou cerca de 15,9 GiB em `sts_correlated_connections`, 11,7 GiB em `sts_health_sync` e 5,7 GiB em `sts_correlate_endpoints`. São tópicos de correlação e saúde do servidor; não são uma cópia de cada gráfico. Há configurações por tópico diferentes da retenção global: por exemplo, `sts_health_sync` tem retenção temporal -1, enquanto outros tópicos usam 24h. A retenção por bytes e o tamanho/rotação de segmentos também importam.

Por isso, não se deve extrapolar indefinidamente +6 GiB/dia nem alterar globalmente a retenção do Kafka como uma “otimização dos dashboards”. Se esse crescimento persistir, acompanhe atraso dos consumidores, rotação/remoção de segmentos e uso por tópico; envolva o suporte SUSE antes de alterar políticas internas. A leitura de configuração e de diretórios foi feita sem alterar o broker ou remover dados.

Filesystem, PVC e armazenamento físico são medidas diferentes

O PVC de métricas de 50 GiB foi rastreado pelo CSI Harvester até o volume Longhorn. Havia uma réplica, estado saudável e aproximadamente 2,74 GiB em `actualSize`, enquanto o filesystem mostrava cerca de 0,94 GiB em uso. Blocos alocados, snapshots, descarte e camadas de virtualização podem produzir diferenças; essa diferença isolada não prova erro na métrica do filesystem. A causa exata de cada bloco não foi determinada. Em um cliente, inclua réplicas, snapshots, backups e folga do datastore no cálculo físico.

5. Estimativa incremental de métricas

Foram contadas cerca de **6101 séries ativas** das fontes do projeto. Com intervalo de 30 segundos, isso equivale a aproximadamente **203 amostras/s**, ou **17,57 milhões/dia**, supondo séries estáveis e intervalos regulares. A ingestão global bruta do VictoriaMetrics foi aproximadamente 3664 amostras/s numa janela de 273 segundos. A proporção de 5,6% é indicativa: não é uma atribuição exata após deduplicação e pode variar com churn e aplicações.

Os arquivos de dados e índices do VictoriaMetrics representavam aproximadamente 1,81 byte por amostra armazenada no conjunto observado. Compressão varia com nomes, labels, churn e repetição; não use esse valor como garantia para outro cliente.

Para planejamento, o quadro abaixo usa **cenários de 2 a 8 bytes/amostra**, não um compromisso de compressão da SUSE. Estima somente dados e índices adicionais deste fluxo, antes de folga operacional e replicação:

Retenção	Cenário 2 bytes/amostra	Cenário 8 bytes/amostra
14 dias	0,46 GiB	1,83 GiB
30 dias	0,98 GiB	3,93 GiB
90 dias	2,95 GiB	11,78 GiB

Fórmula: $\text{séries} \times 86400 \div \text{intervalo_segundos} \times \text{dias} \times \text{bytes_por_amostra} \div 1024^3$. Acrescente margem para compaction, caches, crescimento e operação; depois considere a redundância do servidor e as réplicas do storage. A retenção real do servidor deste laboratório continua sendo um mês. O cenário de 90 dias não foi implantado nem testado.

6. Disco temporário e indisponibilidade do destino

Cada vmagent tem uma fila limitada a 1024 MiB por URL, em emptyDir limitado a 1280 MiB. No laboratório são 22 instâncias: até 22 GiB de filas configuradas, distribuídos pelos nós, e até 27,5 GiB de limites desses volumes temporários. Isso **não estava ocupado**: a métrica de backlog era zero. O request de ephemeral storage é uma reserva diferente; não é um PVC.

Se o destino parar, a fila pode crescer, atingir o limite e descartar dados. emptyDir não sobrevive à substituição do Pod. O Collector KubeVirt tem fila em memória de 128 requests; esse número não representa bytes. Os monitores de backlog/descarte ajudam a detectar o problema. Uma interrupção longa sem perda exige uma decisão específica de durabilidade e capacidade; este pacote não promete armazenamento offline ilimitado.

7. Otimizações implementadas

- Allowlist de famílias efetivamente utilizadas, antes do envio; evita coletar indiscriminadamente todos os exporters.
- Intervalo padrão de 30s e timeout de 10s; sem scrape de 1s para simular tempo real.
- Desativação de loadavg, time e stat no node-exporter: suas famílias já eram descartadas e não eram usadas pelos dashboards/monitores. Mantidos CPU, memória, filesystem, disco, rede e vmstat. Aplicação com canário e rollout de um nó por vez.
- Inventário de Pods paginado, atualização de 30s e expiração de 90s; consultas apenas de leitura, sem um processo de inventário por nó.
- Filas, batch, memória, requests/limits e Jobs com duração limitada. Nenhum publicador fica consultando indefinidamente o servidor.
- Deduplicação de filesystems e seleção de partições operacionais; sem somar bind mounts e discos loop.
- Uma instalação central para os cinco dashboards; métricas compartilhadas entre painéis. Sem duplicar a instalação do Agent ou Collector KubeVirt.

A economia específica de remover os três coletores internos não foi isolada em benchmark. Essa mudança reduz trabalho desnecessário, mas não justifica prometer uma redução numérica de CPU/RAM. Intervalo de 60s poderia reduzir amostras de gauges, porém exige rever taxas, janelas e monitores; não foi aplicado globalmente porque alteraria a resolução validada.

8. Controles de segurança e limites

Controle	Implementação / atenção no cliente
Credenciais	Secrets existentes; ingestão separada de publicação. Nenhuma credencial no Git/ZIP.
Publicadores	Permissões de dashboards ou monitores; sem administrador global. Monitor import exige também delete-monitors na versão validada; o fluxo apply não executa exclusão.
Kubernetes RBAC	Inventários/descoberta somente leitura dos recursos necessários. Coletores locais sem automount de token de ServiceAccount.
Host	node-exporter lê /proc, /sys e raiz do host por mounts read-only; precisa desse acesso para medir o SO. Host networking e hostPath não atendem automaticamente ao perfil Pod Security Restricted. Avaliar exceção no namespace.
Container	Non-root, sem privilege escalation, capabilities removidas, root filesystem read-only e seccomp nos componentes de longa duração. Conferir as políticas de admissão do destino.
Rede	Endpoints locais de host/etcd em loopback. Traefik 9100 sem exposição pública; regra opcional limitada a coletores, somente após revisar a isolação existente.
TLS	Validação de CA e SAN; sem insecureSkipVerify. CA do navegador não substitui a CA dos containers. Renovação das CAs copiadas exige procedimento.
Imagens	Versões explícitas. O publicador valida SHA-256 da CLI. Para produção/air gap, usar registry e imagem aprovados, idealmente digest, com o processo de assinatura/scan da organização. Não foi feita certificação independente de ausência de CVEs.
Dados	Labels com cardinalidade limitada; não adicionar payloads, tokens, URLs com IDs livres ou dados pessoais às métricas.
Notificações	SMTP global e regras manuais, por decisão do usuário. Sem credencial adicional de notificações no chart.

O laboratório apresentou filas vazias, alvos saudáveis e baixo consumo no período medido. Isso não garante impacto zero sob qualquer carga ou indisponibilidade. O aceite deve incluir carga representativa, recursos livres no nó, ausência de OOM/throttling persistente, duração de scrapes abaixo do intervalo, ausência de perdas e crescimento de storage compatível com a retenção.

9. Como repetir a medição no cliente

1. Antes dos charts, registre por pelo menos um período representativo CPU/memória dos nós e servidor, tamanho dos PVCs, ingestão, duração de consultas e retenção. Guarde horários e versões.
2. Instale em um cluster canário. Repita a mesma carga e as mesmas janelas; separe os containers adicionais dos existentes.
3. Compare 24h, sete dias e um ciclo completo de retenção. Não reduza limites apenas porque o primeiro scrape consumiu pouco.
4. Acompanhe os monitores de fonte ausente, fila/descarte, PVC disponível/previsão e latência de disco. Investigue saturação e consumidores antes de aumentar recursos indiscriminadamente.
5. Se uma expansão for necessária, confirme suporte da StorageClass, amplie o PVC e mantenha os valores do servidor consistentes. Apenas mudar o values não amplia automaticamente um PVC existente. Siga o [procedimento de retenção e expansão da SUSE](#).

Comandos de leitura, ajustando o contexto e o namespace:

```
kubectl top pods --kubeconfig "$OBS_KCFG" --context observability \
  -n suse-observability-agent --containers
kubectl top pods --kubeconfig "$OBS_KCFG" --context observability \
  -n suse-observability --containers
kubectl get pvc --kubeconfig "$OBS_KCFG" --context observability \
  -n suse-observability
kubectl get events --kubeconfig "$OBS_KCFG" --context observability \
  -n suse-observability-agent --sort-by=.lastTimestamp
```

No explorador de métricas SUSE, use a janela escolhida e confira os labels reais:

```
sum by (cluster_name) (vmagent_remotewrite_pending_data_bytes{job="sre-collector"})
sum by (cluster_name)
  (increase(vmagent_remotewrite_samples_dropped_total{job="sre-collector"}[30m]))
max by (cluster_name,node,job,instance) (scrape_duration_seconds{job=~"sre-node|sre-etcd|traefik"})
otelcol_exporter_queue_size{service_name="sre-otel-collector"}
max by (persistentvolumeclaim)
  (kubernetes_kubelet_volume_stats_used_bytes{namespace="suse-observability",cluster_name="observability-cluster"})
```

Ausência de uma série de erro não prova zero erros. Confirme que o próprio collector está presente e atualizando antes de interpretar silêncio como saúde.